

University of Science and Technology of Hanoi



GROUP PROJECT

**VISION LANGUAGE MODELS
FOR AUTOMATIC
DENTAL REPORT GENERATION**

Student Name

Nguyen Hoang Tung

Student ID

22BA13388

Hanoi, January 2026

Abstract

Panoramic dental X-ray images are widely used in dental clinics. A single image can show the teeth, jaw area, and several dental conditions. In practice, the dentist not only reads the image, but also writes the findings into a clear report.

This thesis studies Vision-Language Models (VLMs) for dental report generation from panoramic X-ray images. The goal is not to replace dentists. The generated report is treated as a first draft. A trained dentist still needs to check the image, edit the report, and make the final clinical decision.

The current stage focuses on data construction. A PAN924-based instruction dataset is built from ground-truth annotations [1]. Each sample keeps the tooth FDI number, the dental condition label, and the image path. Short clinical comments and summaries are then added for report-style training. Each panoramic image is prepared in five views: one full image and four regional crops. This design keeps both the full mouth context and clearer regional tooth details.

The dataset is first kept in a common format. It is then converted into several model formats, including Qwen, LLaVA, InternVL, Phi, and PaliGemma. This matches the topic name, because the thesis studies several VLMs instead of a single model. A later experiment can compare these VLMs with Faster R-CNN based models. The comparison will focus on report text quality and tooth-level finding correctness.

For now, the main contribution is the data preparation stage. It uses the existing PAN924 annotations, creates five-view image samples, writes report text from fixed templates, balances the rare dental conditions, checks the final data, and exports the dataset for different model families. The training and evaluation setup is also prepared for the next stage.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Problem Statement	5
1.3	Objectives	5
1.4	Literature Review	6
2	Methodology	8
2.1	Overview of the dataset construction process	8
2.1.1	Raw Image Collection	8
2.1.2	Common Report Schema and Prompt Templates	9
2.1.3	Clinical Text Generation from Ground-Truth Labels	10
2.1.4	Validation and Quality Control	10
2.1.5	Final Common Dataset and Label Set	11
2.1.6	Balancing the Rare Conditions	11
2.2	Prepared Outputs for Later Model Experiments	12
3	Experimental Setup	13
3.1	Training Setup	13
3.2	Evaluation Metrics	14
4	Results	16
A	Prompts	17

List of Figures

2.1	Overview of Phase 1: Create image views	9
-----	---	---

List of Tables

2.1	Current PAN924 VLM dataset size	9
2.2	Task types in the common report dataset	10
2.3	Dental condition labels used in the final dataset	11
2.4	Oversampling of the rare conditions in the training split	12
2.5	Sample counts before and after balancing	12
3.1	Training hyperparameters shared by all five models	14
3.2	Groups of evaluation metrics	15

Chapter 1

Introduction

1.1 Background and Motivation

The panoramic dental X-ray, also called an orthopantomogram or OPG, is a common image in dentistry [1]. One image can show the teeth, jawbone, and other nearby structures. Dentists use it for treatment planning, checking missing or impacted teeth, reviewing restorations, and observing dental conditions across the mouth. Because the image covers a wide area, it is useful as an overview image before more detailed treatment decisions are made.

Reading an OPG requires careful work. The dentist first looks at the whole image to understand the general condition of the mouth. After that, smaller regions and individual teeth are checked. For each tooth, the dentist may need to consider the tooth number, visible restorations, caries, implants, residual roots, third molars, or other marked conditions. These findings then need to be written in a form that can be used for treatment planning or further discussion.

This report writing step is important, but it is also repetitive. Many dental reports follow a similar structure. The content changes from patient to patient, but the writer still has to list tooth-level findings and summarize them clearly. In a busy clinic, this can take time, especially when many images are reviewed in the same day. A system that prepares an initial report could reduce part of this repeated writing work.

This thesis starts from that practical problem. The aim is to prepare a first draft of a dental report from an OPG image. The output should not be used as a final diagnosis by itself. It is only a support result, and a dentist must still review it. This point is important because medical images can be difficult to interpret, and final clinical decisions require professional responsibility.

The recent progress of Vision-Language Models (VLMs) makes this direction possible. A VLM can take an image together with a text instruction and produce a text answer. This is useful for report generation because the expected output is not only a class label. The model may need to list teeth, describe conditions, and write a short summary.

At the same time, object detection models such as Faster R-CNN remain strong for localizing objects in images. For this reason, a fair comparison between VLM-style outputs and Faster R-CNN based outputs is also useful.

The title uses “Vision Language Models” in plural because the work is not limited to one model. The data pipeline is prepared so that different VLMs can be trained or tested on the same task. Their results can later be compared with Faster R-CNN based models. Therefore, the thesis is not only about producing one final text output. It also

studies whether VLMs are useful for dental image understanding and structured report generation.

1.2 Problem Statement

A panoramic dental image contains many small details. A single OPG can include up to 32 teeth. Each tooth may have a different condition, such as caries, restoration, implant, residual root, or impacted third molar. The model must deal with both image-level information and tooth-level information. This makes the task harder than a simple image classification problem.

For a VLM, this is not an easy image-to-text task. If the model only receives the full image, small tooth-level details may be missed. The teeth are close to each other, and some findings occupy a small part of the image. If the model only receives small crops, it may lose the full context of the mouth. For this reason, the dataset should support both full-image and regional-image learning.

Another issue is that the desired output is not a single label. A report-like output should contain tooth numbers, condition names, and short written comments. It should also keep the structure clear enough for later evaluation. If the answer is only free text, it is difficult to check whether the model has listed the correct tooth or the correct condition. A JSON-based target format is used in this project to make the output easier to validate.

A second technical problem is the training format. Qwen, LLaVA, InternVL, Phi, and PaliGemma do not all use the same file structure. If the dataset is written only for one model, it is harder to compare different models fairly. For this reason, the dataset is first built in one common format, then converted into model-specific formats. The common dataset works as the stable version of the data, while the converted datasets serve the needs of each model family.

The last problem is report text. The original PAN924 annotations contain useful labels, but they are not written as clinical report sentences. For VLM training, the target answer should include both structured labels and readable text. This thesis handles that step by generating short clinical comments and summaries from the fixed ground-truth labels. The generated comments do not add new medical findings. They only express the existing labels in a report-like form.

1.3 Objectives

The objective of this stage is to build the data foundation for later VLM experiments. The work is limited to data construction and dataset conversion. Model training, model comparison, and final evaluation are left for the next stages of the thesis.

The specific objectives for this stage are:

1. Build a clean instruction dataset from the PAN924 panoramic dental X-ray dataset.
2. Keep the dataset based on ground-truth labels, including FDI tooth numbers and dental condition labels.
3. Create five image views for each original OPG image: one full image and four regional crops.
4. Generate report-style comments and summaries from fixed clinical templates.
5. Validate the final dataset to make sure the image paths, labels, tasks, and output schemas are consistent.

6. Convert the common dataset into training formats for several VLM families, so later experiments can compare different models on the same data split.

7. Prepare a balanced training set that gives the rare dental conditions a stronger learning signal, while keeping the validation and test sets unchanged.

These objectives are chosen to keep the project realistic at bachelor level. The main focus is not to build a large clinical system immediately. The focus is to prepare a reliable dataset and a clear pipeline, so that later experiments can be run in a controlled way.

1.4 Literature Review

Medical Image Report Generation

Medical image report generation has been studied for several years. Many systems use an image encoder and a text decoder. The image encoder reads visual features from the image. The text decoder then generates a report. This design is similar to image captioning, but the medical setting is stricter. The report needs to mention clinically useful findings, not just describe visible objects.

Later methods used attention and Transformer-based models. R2Gen is one example. It used a memory-based Transformer design for radiology report generation and tested it on IU X-Ray and MIMIC-CXR [2]. This type of work shows that report generation benefits from both image features and language patterns. A radiology report often uses repeated medical phrases, so the model needs to learn both visual evidence and report style.

Most well-known work in this area still focuses on chest X-rays. Dental panoramic images are different. A chest X-ray usually has large anatomical regions, while an OPG contains many small tooth regions in one image. The output also needs tooth numbers and tooth-level conditions. This makes direct transfer from general radiology report generation less simple.

For this thesis, the main lesson from previous work is simple: the model needs data from the target medical domain. A general model may write fluent sentences, but it may still miss dental details. This is why the dataset construction step is important before training or comparing VLMs. The dataset must show the model how dental images, tooth labels, and report text are connected.

Vision-Language Models in Dental Radiology

Vision-Language Models take an image and a text instruction as input. Then they return a text answer. LLaVA is a clear example of this idea, because it connects a vision encoder with a large language model and uses visual instruction tuning [3]. In medicine, VLMs are also studied for report generation and visual question answering [4].

This instruction-based setup is useful for dental reporting. The user can ask the model to return a specific structure, such as a list of visible teeth and their conditions. The answer can include both labels and short natural-language comments. This is different from a detector, which normally returns bounding boxes and class labels.

In dental radiology, VLMs are still newer than detection models. A VLM may be useful because it can return a structured answer, not just a class label or a bounding box. For example, it can list teeth, give condition names, and write a short summary in one

response. However, this does not mean that a VLM is automatically better. It still needs proper data and careful evaluation.

Different VLMs also have different input formats. Qwen-style data, LLaVA-style chat data, InternVL conversations, Phi prompts, and PaliGemma samples are not identical. To keep the data consistent, this thesis uses one common dataset as the main version. The converted files are generated from that dataset, so model comparison is cleaner. This is also useful because the same training and testing splits can be reused for different model families.

Tooth Detection on Panoramic X-rays

Faster R-CNN is a common baseline for object detection. In dental panoramic X-rays, it can detect tooth boxes, tooth numbers, or dental condition classes. Tuzoff et al. used Faster R-CNN for tooth detection and reported high detection performance on panoramic radiographs [5].

This type of model is strong when the output is a fixed label and a bounding box. It is also easier to evaluate with standard detection metrics. For example, the model can be judged by whether the tooth location and class are correct. This makes Faster R-CNN a practical baseline for tooth-level tasks.

However, a detection model does not naturally write a clinical report. It can say where a tooth is and what class it belongs to, but it does not directly produce a readable paragraph or a structured report summary. A VLM can produce text, so it may be better for report-style output. At the same time, the VLM output is harder to evaluate because it mixes recognition and language generation.

Because of that, Faster R-CNN is useful as a comparison point. Later experiments can compare VLM outputs with Faster R-CNN based results. The comparison can focus on tooth-level correctness, condition labels, and report text quality. This helps separate two questions: whether the model recognizes the dental findings, and whether it can express them in a useful report format.

Chapter 2

Methodology

2.1 Overview of the dataset construction process

2.1.1 Raw Image Collection

The raw data comes from the PAN924 panoramic dental X-ray dataset [1]. It contains 924 panoramic images. Each image includes tooth annotations with the FDI tooth number and the dental condition label.

The original images are stored in the `images/` folder. The main annotation file is `annotations/dataset_final_v2.json`. This file is used as the source for the VLM dataset. No new disease labels are created by hand in this step. The labels come from the existing ground-truth annotation file.

The first script in the pipeline is `build_5view_common_dataset.py`. It reads the original annotations, loads the source images, and builds the common training data. This common dataset is the main dataset of the thesis. The model-specific datasets are created from it later.

For each panoramic image, the pipeline creates five image views:

- one full panoramic image;
- one upper-left image-space crop;
- one upper-right image-space crop;
- one lower-left image-space crop;
- one lower-right image-space crop.

The regional crops are not random. The image is split using the image width and the tooth annotation positions. The vertical split uses the median position of upper and lower tooth annotations. A 10% overlap is added in both directions to keep useful tooth information near the crop borders.

The region names are image-space names. For example, `image_upper_left` means the upper-left crop in the image. It does not mean the anatomical left side of the patient.

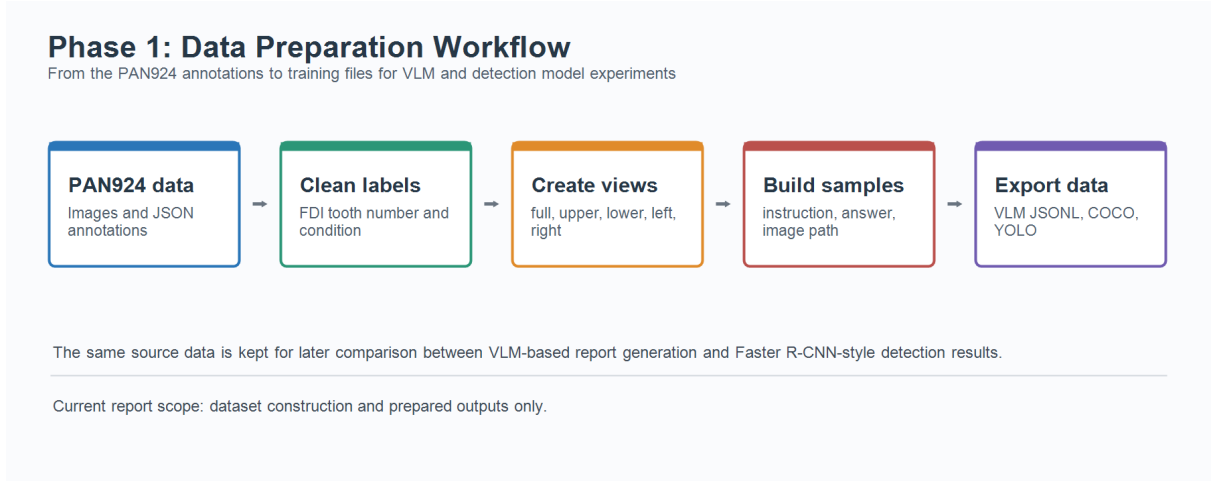


Figure 2.1: Overview of Phase 1: Create image views

Table 2.1: Current PAN924 VLM dataset size

Split	Source images	Samples	Description
Train	739	3695	Five views per image
Validation	92	460	Five views per image
Test	93	465	Five views per image
Total	924	4620	924 full reports and 3696 regional reports

2.1.2 Common Report Schema and Prompt Templates

The current pipeline does not build open-ended question-answer data. It builds a common report dataset for VLM training. This is a better fit for the current thesis stage because the output needs to look like a short dental report, not like a question-answer pair.

Each sample is stored as one JSONL row. The row keeps the image path, source image name, view name, task name, prompt, target answer, labels, and metadata. The answer is always stored as valid JSON. This makes the dataset easier to inspect by script and easier to convert later.

There are two task types. The first task is **full_quadrant_report**. It uses the full panoramic image. The target output contains the four image-space regions used in the crop step. Each region has a list of visible annotated teeth, the FDI number, the condition code, the condition name, and one short comment. The full report also has an overall summary.

The second task is **regional_report**. It uses one cropped image region. The target output contains the region name, the visible annotated teeth in that crop, and one short comment for the region.

The prompt is kept short on purpose. It tells the model the image type and the JSON schema that must be returned. Long medical instructions are not used at this stage because the first goal is to make a stable dataset. The clinical meaning comes from the ground-truth labels, not from a free-form prompt.

Table 2.2: Task types in the common report dataset

Task	Image input	Target output
<code>full_quadrant_report</code>	Full panoramic image	Four-region JSON report and summary
<code>regional_report</code>	One regional crop	Tooth list and region comment

2.1.3 Clinical Text Generation from Ground-Truth Labels

The report text is generated from the annotation labels. No external AI model or API is used in this step. This is important because the dataset should not add new findings that are not in the original annotations.

The script `generate_clinical_reports.py` reads the raw common JSONL files from the raw subfolder of `common`. Then it writes the final JSONL files back to `common`. The script only changes the `comment` and `summary` fields. It does not change the image path, split, task name, tooth number, condition label, or JSON structure.

The text generation rule is simple. If a region has abnormal findings, those findings are mentioned first. Teeth with the same condition are grouped together. Healthy teeth are not listed one by one when abnormal teeth already exist. If a region has only healthy annotated teeth, the comment says this clearly. If a region has no annotated teeth, the comment also says this clearly.

For example, if a region contains caries on one tooth and a restoration on another tooth, the generated comment has this style:

This region shows caries on tooth 33 and a restoration on tooth 34. The remaining annotated teeth are healthy.

This text is not used as a final clinical diagnosis. It is used as a training target for report-style VLM output. In a real dental setting, the original X-ray and any generated report must still be checked by a qualified dentist.

2.1.4 Validation and Quality Control

After the common dataset is generated, the validation script checks whether each JSONL row has the required fields. It also checks the image paths, task names, output JSON schema, condition labels, and split information.

For full-image reports, the validator checks that all four region keys are present. It also checks that the target teeth match the saved labels for each region. For regional reports, it checks that the region name and tooth list in the target output match the label fields.

The validation result shows `ok: true` and `error_count: 0`. The final common dataset contains 924 unique source images and 4620 samples. These include 924 full-image report samples and 3696 regional report samples. The split is image-based, so one source image does not appear in more than one split.

Several samples are also inspected manually. This is mainly to see whether the comments match the tooth labels and whether the crops are useful. This manual check is not the same as expert dental review. It is only a practical check for dataset consistency at this stage.

2.1.5 Final Common Dataset and Label Set

The final common dataset is stored in `vlm.report.dataset/common/`. This folder is treated as the main dataset. It contains `train.jsonl`, `val.jsonl`, and `test.jsonl`. It also contains the copied full images, the four regional crops for each image, and metadata files such as the build summary, split report, crop manifest, clinical generation summary, and validation report.

The dataset uses 12 condition codes after label cleaning. Some rare or overlapping labels are remapped before the final files are written. For example, `Ri`, `RiM`, and `TeM` are mapped to `Te`. The label `I` is mapped to `M3i`. This keeps the label set easier to handle during training and evaluation.

Table 2.3: Dental condition labels used in the final dataset

Code	Condition name
H	healthy
C	caries
R	restored
Te	endodontic treatment
Im	implant
Rr	residual root
M3i	impacted third molar
M3f	developing third molar
CpuM	prosthetic crown
Dc	crown destruction
Di	incisal or occlusal wear
P	pontic

2.1.6 Balancing the Rare Conditions

The dental conditions are not evenly distributed. Healthy teeth and restorations appear very often. Other conditions are rare. In the training split, crown destruction (`Dc`), implant (`Im`), pontic (`P`), residual root (`Rr`), and developing third molar (`M3f`) each appear far less than the common labels. If the data is used as is, the model sees these rare conditions only a few times. It can then learn to ignore them.

To reduce this problem, the script `rebalance_common.py` builds a balanced training set. It does not create new images. It only repeats some existing samples, so the rare conditions are seen more often. The new set is written to `common_balanced/`, next to the common dataset.

The balancing uses two simple ideas together. First, only the regional crops are repeated, never the full reports. A crop lists about eleven teeth, while a full report lists about twenty-one. In a crop, a rare tooth is a larger part of the answer, so the learning signal is stronger. Second, each rare condition gets a copy multiplier based on how rare it is. A rarer condition is copied more times.

A few rules stop the copying from going too far. The multiplier is capped at eight, so no class grows without limit. When one sample has more than one rare condition, the multiplier is the largest one, not the product. The copies are spread over all crops that

contain the condition, so a single image is not repeated too many times. The validation and test sets are copied without any change, so the evaluation stays honest.

Table 2.4: Oversampling of the rare conditions in the training split

Code	Condition	Multiplier	Teeth before	Teeth after
Dc	crown destruction	7.89	174	981
Im	implant	8.00	199	1081
P	pontic	4.00	362	1178
Rr	residual root	3.97	332	1109
M3f	developing third molar	3.34	398	959

After balancing, the training split grows from 3695 to 5817 rows. This is about 57% more rows. The validation and test splits do not change. Table 2.5 shows the sample counts.

Table 2.5: Sample counts before and after balancing

Split	Before balancing	After balancing
Train	3695	5817
Validation	460	460
Test	465	465

The balanced training set is kept as a separate option. Later experiments can train on the original training split or on the balanced one, and then compare the two.

2.2 Prepared Outputs for Later Model Experiments

The common dataset is model-independent. After it is validated, the conversion script writes the same data into model-specific folders. This keeps the source data fixed while allowing later experiments with different VLM families.

At the current stage, converted files are prepared for Qwen, LLaVA, InternVL, Phi, and PaliGemma. Each converted folder keeps the same train, validation, and test counts: 3695 training samples, 460 validation samples, and 465 test samples. This makes later comparison easier because the models can be trained or tested on the same split.

The conversion step changes only the outer data format. For example, Qwen uses a message format with an image list, LLaVA uses multimodal chat-style messages, InternVL uses conversation-style rows, Phi uses a prompt and response format, and PaliGemma uses prefix and suffix fields. The image paths, target report content, split names, and metadata still come from the common dataset.

This report currently stops at the data preparation stage. Model training, model comparison, and final result analysis are planned for the next stage. The prepared dataset can later be used to compare VLM report outputs with Faster R-CNN-style detection results, but that experiment has not been completed yet.

Chapter 3

Experimental Setup

Model training and evaluation are the next stage of the thesis. The training scripts and the evaluation script are already prepared, so this chapter describes the planned setup. No model has been trained yet, and no results are reported here.

3.1 Training Setup

Five VLM families are planned for training: Qwen, LLaVA, InternVL, Phi, and PaliGemma. All of them use the same training settings. This is done on purpose, so the later comparison between models is fair. Only the model name, the output folder, and the image resolution option change between models.

The models are fine-tuned with LoRA. LoRA trains a small number of extra weights and keeps the base model mostly frozen. This is cheaper and fits on a single GPU. On a 24 GB RTX 3090, the base model is loaded in 4-bit, which is the QLoRA setting, to save memory. On a 40 GB A100, the model is trained in bf16 without quantization. The hyperparameters are the same in both cases, so the two machines give comparable runs.

Table 3.1: Training hyperparameters shared by all five models

Setting	Value
Fine-tuning method	LoRA (QLoRA 4-bit on the RTX 3090)
LoRA rank	8
LoRA alpha	32
LoRA dropout	0.1
Target modules	all linear layers
Vision encoder	frozen
Epochs	2 (best checkpoint chosen by validation)
Learning rate	1e-4
LR scheduler	cosine
Warmup ratio	0.05
Weight decay	0.1
Batch size per device	1
Gradient accumulation	16 (effective batch size 16)
Max sequence length	4096
Max image pixels	802,816
Eval / save interval	every 100 steps
Random seed	924

The best checkpoint is not always the last one. The training saves a checkpoint every 100 steps and checks the validation set. The checkpoint with the best validation score is then used for the final test. This avoids picking a model that has already started to overfit.

3.2 Evaluation Metrics

The evaluation uses one script, `eval_report.py`, for all models. It compares the predicted JSON report with the gold JSON report. The basic unit is one tooth in one region. For each unit, the script checks the FDI number and the condition label.

The metrics are grouped in a simple way. The detection metrics ask if the model finds the right teeth. The classification metrics look at whether a found tooth has the correct condition. The joint metrics combine the two. The last group checks the text quality and the JSON format. Table 3.2 lists the groups.

Table 3.2: Groups of evaluation metrics

Group	What it measures
Detection	Whether the right teeth are found by FDI number: precision, recall, F1, plus the rate of invented teeth and missed teeth.
Classification	For a tooth that is found, whether the condition is correct: per-condition and averaged precision, recall, and F1.
Joint	Whole-tooth accuracy, where the FDI number and the condition are both correct, and the exact report match rate.
Generation	The share of outputs that are valid JSON, and the ROUGE-L score on the comment and summary text.

These metrics cover both parts of the task. The detection and classification metrics show if the model finds the correct dental findings. The generation metrics show if the output is valid and readable. The detection and classification metrics can also be used for the Faster R-CNN baseline, so the recognition part can be compared on the same ground.

Chapter 4

Results

No model results are reported at this stage. The current result is the completed PAN924-based VLM report dataset and its validated converted formats.

Appendix A

Prompts

Prompt Template for Full Panoramic Report

```
<image>
This is a full panoramic dental radiograph.
Divide the image into four image-space regions: image_upper_left,
    image_upper_right, image_lower_left, and image_lower_right.
For each region, list the visible annotated teeth using FDI notation
    and report each tooth condition.
Return only valid JSON with this schema:
{"regions":{"image_upper_left":{"teeth":[{"fdi":"string","condition":"
    string","condition_name":"string"}],"comment":"string"},"
    image_upper_right":{"teeth":[{"fdi":"string","condition":"string","
    condition_name":"string"}],"comment":"string"},"image_lower_left":{"
    teeth":[{"fdi":"string","condition":"string","condition_name":"
    string"}],"comment":"string"},"image_lower_right":{"teeth":[{"fdi":"
    string","condition":"string","condition_name":"string"}],"comment":"
    string"}},"summary":"string"}
```

Listing A.1: Prompt template for full panoramic dental report

Prompt Template for Regional Report

```
<image>
This is the {region} crop of a panoramic dental radiograph.
Identify the visible annotated teeth using FDI notation and report each
    tooth condition.
Return only valid JSON with this schema:
{"region":"string","teeth":[{"fdi":"string","condition":"string","
    condition_name":"string"}],"comment":"string"}
```

Listing A.2: Prompt template for regional dental report

References

- [1] Interdisciplinary Research Group in Digital Dentistry, “InReDD-Dataset-PAN924 v1.0.0,” PhysioNet, 2025, accessed 2026-06-09. [Online]. Available: <https://physionet.org/content/inredd-dataset-pan924/1.0.0/>
- [2] Z. Chen, Y. Song, T.-H. Chang, and X. Wan, “Generating radiology reports via memory-driven transformer,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 1439–1449. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.112/>
- [3] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.08485>
- [4] I. Hartsock and G. Rasool, “Vision-language models for medical report generation and visual question answering: a review,” *Frontiers in Artificial Intelligence*, vol. 7, p. 1430984, 2024. [Online]. Available: <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2024.1430984/full>
- [5] D. V. Tuzoff, L. N. Tuzova, M. M. Bornstein, A. S. Krasnov, M. A. Kharchenko, S. I. Nikolenko, M. M. Sveshnikov, and G. B. Bednenko, “Tooth detection and numbering in panoramic radiographs using convolutional neural networks,” *Dentomaxillofacial Radiology*, vol. 48, no. 4, p. 20180051, 2019. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6592580/>